

Проблема: Считать слишком долго

Контекст 2004 года (Google): 1 гигабайт — это много. Сеть медленная. Сбои — это норма.

Ключевая идея: Мы перемещаем код к данным, потому что перемещать 1 МБ кода дешевле, чем 1 ТБ данных.

Аналогия: Представьте, что вам нужно посчитать количество букв в каждой книге библиотеки. *Вы можете:*

1. Отнести все книги эксперту (данные перегружают сеть).
2. Разослать экспертов в каждый зал библиотеки (MapReduce).

Контракт MapReduce

MAP: $(K1, V1) \rightarrow \text{список } (K2, V2)$

Видит одну запись. Ничего не знает о других.

SHUFFLE + SORT: Группировка по ключу. Сортировка. Сеть. Диск.

REDUCE: $(K2, \text{Iterable}\langle V2 \rangle) \rightarrow \text{список } (K3, V3)$

Получает ВСЕ значения по ключу ОДНИМ вызовом.

Каждый этап пишет на диск, прежде чем читать дальше

Что убило MapReduce?

ДИСК: Итеративный алгоритм = 20 проходов = 20× запись/чтение.

КОМПОЗИЦИЯ: Два прохода = два Job'a + bash-скрипт.

ЯЗЫК: Java + Writable. 50 строк для WordCount.

ИТОГ: Жигуль. Едет, но хочешь скорость, безопасность и комфорт?
Покупай ...

RDD — это не данные, это рецепт

Внутри:

- Ссылка на родительский RDD
- Функция трансформации
- Метаданные (партиции)

Аналогия

MapReduce — повар готовит и съедает.

RDD — повар записывает рецепт.

Action — «Приготовить сейчас!»

Преимущество

Упал клиент? Не надо переваривать — следуй рецепту.

Ленивые и Активные

ТРАНСФОРМАЦИИ — строят план

- map, filter, flatMap — узкие (без shuffle)
- reduceByKey, join,... — широкие (shuffle!)

ДЕЙСТВИЯ — запускают вычисления

collect, count, take, save, reduce

Ошибка новичка: .collect() на 10 ГБ в драйвер

Пример

```
val rdd = sc.textFile("data")      // ничего

val filtered = rdd.filter(_.contains("error")) // ничего

val mapped = filtered.map(parseLine) // ничего

mapped.count()                     // ПОЕХАЛИ!
```

cache() — половина успеха

БЕЗ cache()

```
val rdd = sc.textFile("logs").map(parseLine)
```

```
rdd.filter(_.error).count() // читает + парсит
```

```
rdd.filter(_.warn).count() // ЧИТАЕТ + ПАРСИТ СНОВА!
```

С cache()

```
val rdd = sc.textFile("logs").map(parseLine).cache()
```

```
rdd.filter(_.error).count() // читает, парсит, кэширует
```

```
rdd.filter(_.warn).count() // из кэша!
```

Правило: множественные действия = cache()

Spark UI — глаза инженера

- Жизненный цикл = набор Job-ов (заранее неизвестно сколько джобов будет в контексте).
- Каждая Job делится на Stages, анализируя их можно понять, где тормозит.
- Каждый Stage состоит из множества Tasks (минимальные единицы вычислений).
- Каждая Task выполняется на одном из Executors, в случае падения, выполняется снова. Можно отследить кто-сколько работал, кто сколько читал-писал и потреблял сборку мусора.
- Конфигурация Executors (количество ядер и памяти на один процесс) определяется при создании Spark-контекста и не может быть изменена в процессе выполнения задач.

Ключевые проблемы

- Данные не влезают в память → spill на диск, OOME
 - Увеличить память
 - Увеличить количество партиций
 - Использовать `.persist(StorageLevel.DISK_ONLY)` вместо `.cache()`
- 99% executors простаивают, один умирает
 - Увеличить количество партиций
 - Хитрое партиционирование
 - Предварительные агрегации

Паттерн 1. Разворот структур

На входе список друзей

- (1, [2,3,4])
- (2, [1,5])

На выходе пары друзей

(1,2), (1,3), (1,4), (2,1), (2,5)

Пример

```
rdd.flatMap { case (user, friends) =>  
  friends.map(friend => (user, friend))  
}
```

Паттерн 2. Фильтр-Агрегация

ПЛОХО

```
rdd.groupByKey()
```

```
.filter(_._1 == "RU")
```

```
.mapValues(_._2.sum)
```

Лучше

```
rdd.filter(_._1 == "RU")
```

```
.groupByKey()
```

```
.mapValues(_._2.sum)
```

Совсем хорошо

```
rdd.filter(_._1 == "RU")
```

```
.reduceByKey(_+_)
```

Что внутри reduceByKey

Локальная сумма в каждой партии

В shuffle — только суммы

Внимание!!! flatMap + groupByKey != MapReduce

groupByKey хоть и дает доступ к итератору, но поднимает все в память

Паттерн 3. Двойная агрегация

Задача: количество чеков + сумма по товару

Вход: (товар, цена)

Выход: (товар, (кол-во, сумма))

```
rdd.map { case (product, price) =>  
  (product, (1, price))    // два счетчика сразу  
}.reduceByKey(_|+|_) // Используем Scalaz
```

Выигрыш:

Один Shuffle вместо двух (данные читаются 1 раз)

Паттерн 4. Борьба с перекосом (Salting)

Проблема: ключ с 500М записей — один редьюсер тонет

rdd

```
.map { case (user, value) => ((user, Random.nextInt(100)), value) } // Добавляем соль
```

```
.reduceByKey(_ + _) // Первичная сумма
```

```
.map{case((user, _), value) => (user, value)}
```

```
.reduceByKey(_ + _) // Первичная сумма
```

Результат: 1 редьюсер → 100 редьюсеров

EDA — вы не знаете схему

EDA (Exploratory Data Analysis) — разведочный анализ данных.

```
sc.textFile("data").take(10).foreach(println)
```

```
// Ага, разделитель |
```

```
val sample = sc.textFile("data").map(_._split("\\|")).take(100)
```

```
// Поле 3 — Int, поле 5 — Double
```

ИТОГ: 2 минуты — и вы работаете с данными

А как получить максимальный аналог MapReduce?

```
rdd.flatMap{case(key, value) => ...} // Получаем коллекцию ключей и значений  
  .repartitionAndSortWithinPartitions(partitioner) // Сортируем по ключу  
  .mapPartitions { iter =>  
    // iter уже отсортирован по ключу внутри партии  
  }
```

Зачем это нужно?

- Полный контроль над партиционированием
- Сортировка на месте, без лишнего shuffle
- Когда нужен итератор по значениям (как в MapReduce)

Пример: Топ-N по ключу без OOM

Задача: для каждого товара топ-5 покупателей (groupByKey — OOM на больших группах)

```
rdd.map(p => ((p.productId, -p.amount), p.userId))
  .repartitionAndSortWithinPartitions(new FirstElementPartitioner(200))
  .mapPartitions { iter =>
    var currentProduct = -1
    var count = 0
    iter.flatMap { case ((productId, _), userId) =>
      if (productId != currentProduct) {
        currentProduct = productId
        count = 1
        List((productId, userId))
      } else if (count < 5) {
        count += 1
        List((productId, userId))
      } else List()
    }
  }
```

```
class FirstElementPartitioner(partitions: Int) extends Partitioner {
  override def numPartitions: Int = partitions

  override def getPartition(key: Any): Int = {
    key match {
      case (fe, _) => (fe % partitions).abs
      case _ => 0
    }
  }
}
```

А что насчет Parquet + Binary?

```
rdd.map { row =>
```

```
  val data = MyData.parseFrom(row.getAs("payload"))
```

```
  ...
```

```
}
```

Итог: пара строк кода и можно работать с десериализованным объектом

Итоги

1. **MapReduce — это история.** Но она научила нас думать парами (ключ, значение)
2. **RDD — не устаревшая технология.** Это контроль там, где абстракции бессильны
3. **10 метрик = 1 строка.** Scalaz + |+| — композируемые агрегации
4. **repartitionAndSortWithinPartitions.** MapReduce с итератором без OOM (условно)
5. **Parquet + Protobuf = зона смерти DataFrame.** Байты, версии, битые записи — только RDD
6. **cache() — половина успеха.** Множественные действия = cache()

Домашнее задание

ФОРМАТ

Вы — архитектор данных. К вам приходит джуниор:

«Я буду все писать на DataFrame, это современно».

ВАША ЗАДАЧА

Придумать задачу, на которой вы докажете, что RDD:

Компактнее — меньше кода

Быстрее — ручные оптимизации добавляют скорость

Надежнее — возможность управлять памятью

Гибче — позволяют сделать то, для чего DataFrame не очень подходит

УСЛОВИЕ: Задача должна быть реалистичной.